

第7章 排序

目录

第一节 排序概述

第二节 插入排序

第三节 交换排序

第四节 选择排序

第五节 归并排序

第一节 排序概述

什么是排序



排序是指将一组数据按照关键字值的大小(递增或者递减)次序进行排列。排序是线性表、二叉树等数据结构的一种基本操作。

作为排序依据的数据项叫关键字。关键字分为两种，能唯一标识一条记录的称为主关键字；标识多条记录的叫次关键字。

排序的分类



内部排序和外部排序：内部排序排序序列完全存放在内存中；

外部排序需要将数据元素存储在外部存储器上。



稳定排序和不稳定排序：稳定排序在依据关键字进行排序后具有相同关键字的数据元素的位置关系与排序前相同，不稳定排序则相反。

排序算法的性能评价



通常从时间复杂度和空间复杂度两个方面评价算法的性能。时间复杂度主要用算法执行过程中的比较和移动次数计算； 排序的空间复杂度主要用外部存储空间的大小来计算。

排序往往处于软件的核心部分，经常被使用，所以其性能的优劣对软件质量的好坏起着重要的作用

插入排序	交换排序	选择排序	归并排序
01	02	03	04

本章中的排序算法均以顺序表为基础进行设计

第二节 插入排序

- 直接插入排序
- 希尔排序

直接插入排序

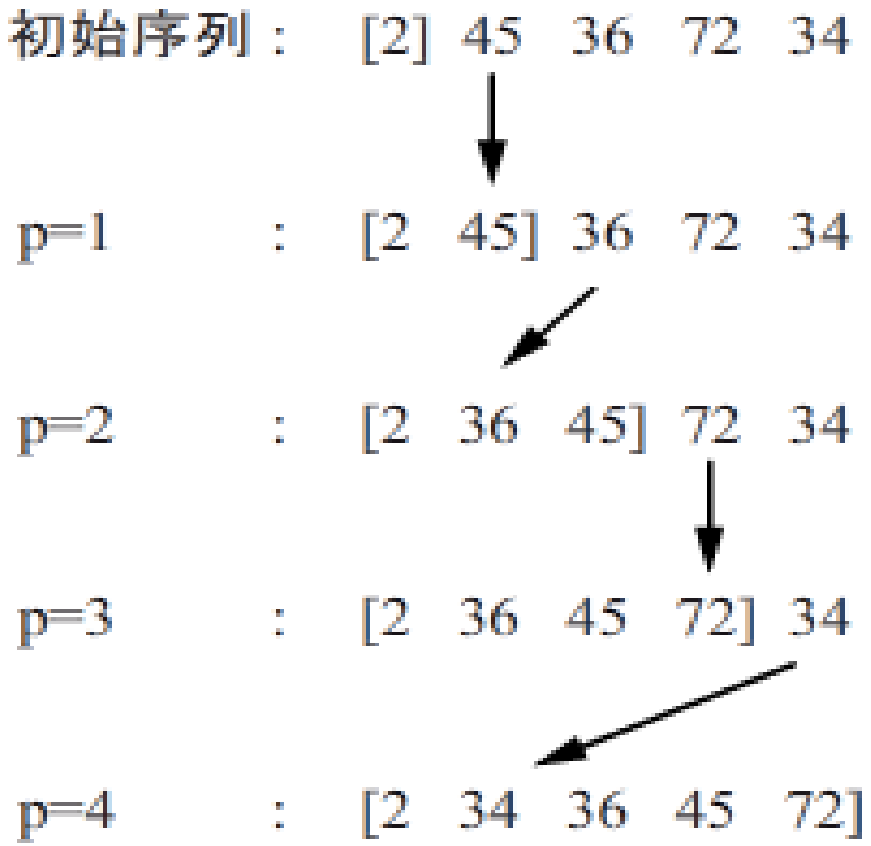
- 1. 直接插入算法的实现
- 直接插入排序是指将一条待排序的记录按照其关键字值的大小插入到已排序的记录序列中的正确位置，依次重复，直到全部记录都插入完成。其主要步骤如下:
- (1) 将list[i]存放在临时变量p中。
- (2) 将p与list[i-1]、list[i-2]、…、list[0]依次比较，若有p<list[j].key(j=i-1、i-2、…、0)，则将list[j]后移一个位置，直到p≥list[j].key为止。当p≥list[j].key时将p插入到list[j+1]的位置。
- (3) 令i=1、2、3、…、n-1，重复步骤(1)、(2)、(3)。



01 02 03 04 GOAL

直接插入排序

- 假设一组待排序的记录的关键字序列为{2,45,36,72,34}，直接插入排序的过程如图所示：



Python实现

```
1  def insertSort(self):
2          # 进行 len-1 次扫描
3          for i in range(1,self.len):
4                  p = self.list[i]
5                  # 比 list[i]大的元素后移
6                  j = i-1
7                  while j>=0:
8                          if self.list[j].key>p.key:
9                                  self.list[j+1] = self.list[j]
10                                 j -= 1
11                         else:
12                                 break
13                 self.list[j+1] = p # 插入
```

算法性能分析

- 1) 时间复杂度
- 有序表中逐个插入的操作进行了n-1趟，每趟的插入操作的时间主要耗费在关键字的比较和数据元素的移动上。
- 在最好情况下待排列的顺序表已按关键字值有序，每趟排序比较一次，移动两次，总的比较和移动次数为3(n-1)； 在最坏情况下待排序的顺序表已按关键字值逆序排列，每趟比较i次，移动i+2次，总的比较和移动次数为$\sum_{i=1}^{n-1}(2i+2)=n^2+n$； 在一般情况下，排序记录是随机序列，第i趟排序所需的比较和移动次数取平均值，约为i次，总的比较和移动次数为$\sum_{i=1}^{n-1} i=\frac{n(n-1)}{2}\approx\frac{n^2}{2}$，因此直接插入排序的时间复杂度为$O(n^2)$。



算法性能分析

- 2) 空间复杂度
- 由于其仅使用了一个辅助存储单元p，所以空间复杂度为O(1)。
- 3) 算法稳定性
- 在使用直接插入排序后具有相同关键字的数据元素的位置关系与排序前相同，因此直接插入排序是一种稳定的排序算法。



2.2 希尔排序

- 希尔排序是D.L.Shell在1959年提出的，又称缩小增量排序，是对直接插入排序的改进算法，其基本思想是分组的直接插入排序。

- 由直接插入排序算法分析可知，数据序列越接近有序时间效率越高，当n较小时时间效率也较高。希尔排序正是针对这两点对直接插入排序算法进行改进。



- (1) 将一个数据元素序列分组，每组由若干个相隔一段距离的元素组成，在一个组内采用直接插入算法进行排序。

- (2) 增量的初值通常为数据元素序列的一半，以后每趟增量减半，最后值为1。随着增量逐渐减小，组数也减少，组内元素的个数增加，数据元素序列接近有序。

2.2 希尔排序

其主要步骤如下。

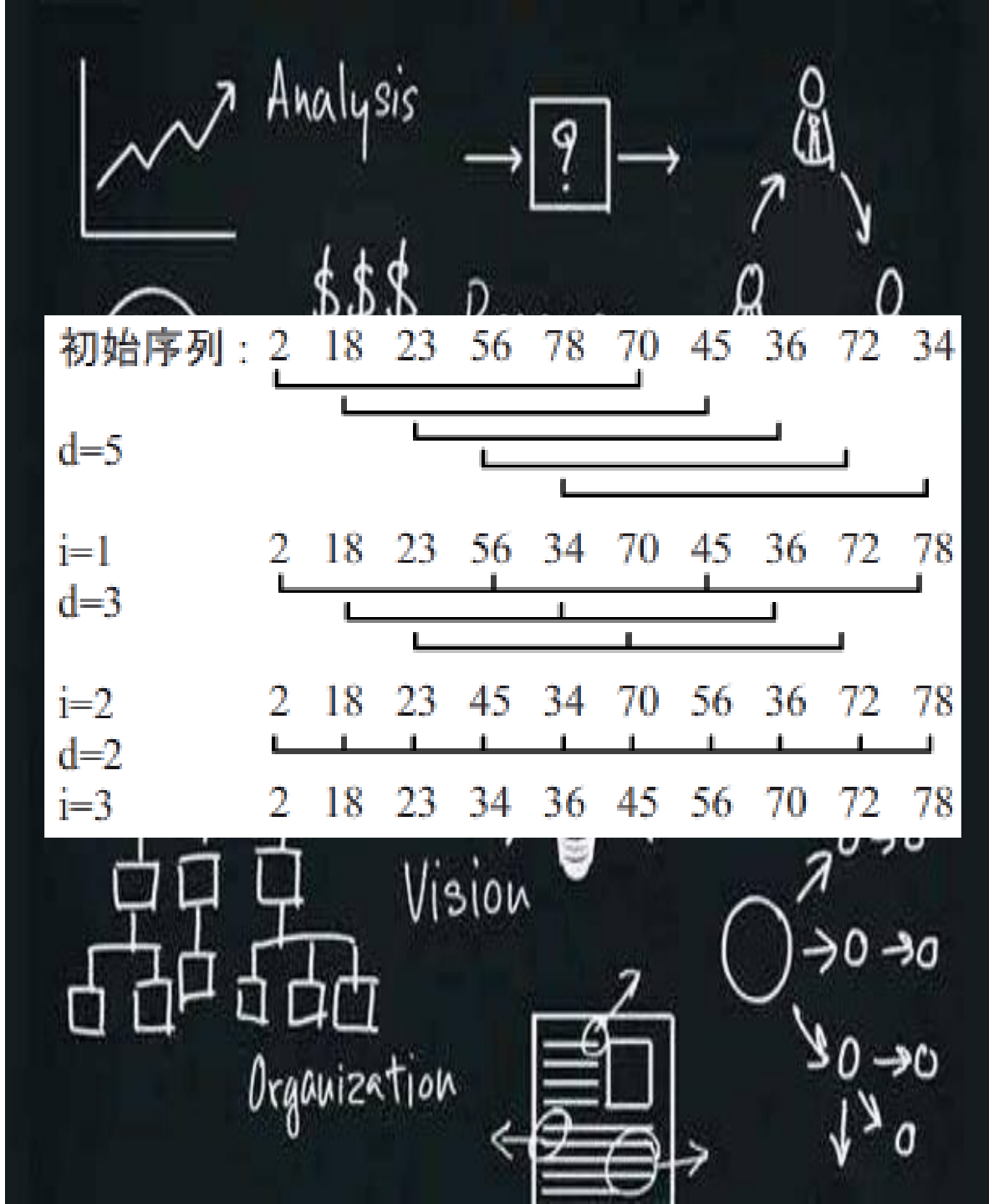
(1) 设定一个增量序列{d_0,d_1，…,1}。

(2) 根据当前增量d_i将间隔为d_i的数据元素组成一个子表，共d_i个子表。

(3) 对各子表中的数据元素进行直接插入排序。

(4) 重复步骤(2)、(3)，直到进行完d_i=1，此时序列已按关键字值排序。

假设一组待排序的记录的关键字序列为{2,18,23,56,78,70,45,36,72,34}，增量分别取5、3、1，则希尔排序的过程如图7.2所示。



Python实现

```
1  def shellSort(self,d):
2          for k in d:
3                  # 在增量内进行直接插入排序
4                  j = k
5                  while j<self.len:
6                          p = self.list[j]
7                          m = j
8                          while m>=k:
9                                  if self.list[m-k].key>p.key:
10                                         self.list[m] = self.list[m-k]
11                                         m = m-k
12                                 else:
13                                         break
14                         self.list[m] = p
15                         j += 1
```

算法性能分析

时间复杂度

希尔排序的关键字比较次数和数据元素的移动次数取决于增量的选择，目前还没有更好的选取增量序列的方法。Hibbard提出了一种增量序列 $\{2^k-1,2^{k-1}-1,\ldots,7,3,1\}$，可以使时间复杂度达到 $O(n^{3/2})$。需要注意的是，在增量序列中应没有除1以外的公因子，并且最后一个增量值必须为1。

空间复杂度

希尔排序仍只使用了一个额外的存储空间p，其空间复杂度为O(1)。

算法稳定性

希尔排序算法在比较过程中会错过关键字相等的数据元素的比较，算法不能控制稳定，因此希尔排序是一种不稳定的排序算法。

第三节 交换排序

- 冒泡排序
- 快速排序

3.1 冒泡排序



- 冒泡排序算法的实现
- 冒泡排序是两两比较待排序记录的关键字，如果次序相反则交换两个记录的位置，直到序列中的所有记录有序。若按升序排序，每趟将数据元素序列中的最大元素交换到最后的位置，就像气泡从水里冒出一样。其主要步骤如下：

3.1 冒泡排序

其主要步骤如下。

(1) 设交换次数k=1。

(2) 在常数为n的序列{a[0],a[1],…,a[n-1]}中从头到尾比较a[i]和a[i+1]，若a[i].key>a[i+1].key，则交换两个元素的位置，其中，0≤i<n-i。

(3) k增加1。

(4) 重复步骤(2)、(3)，直到k=n-1或者步骤(2)中未发生交换为止。

假设一组待排序的记录的关键字序列为{2,23,18,56,78,70,45,36,72,34}

初始序列：	2	23	18	56	78	70	45	36	72	34
第一趟：	2	18	23	56	70	45	36	72	34	78
第二趟：	2	18	23	56	45	36	70	34	72	78
第三趟：	2	18	23	45	36	56	34	70	72	78
第四趟：	2	18	23	36	45	34	56	70	72	78
第五趟：	2	18	23	36	34	45	56	70	72	78
第六趟：	2	18	23	34	36	45	56	70	72	78



Python实现

```
1  def bubbleSort(self):
2          flag = True
3          i = 1
4          while i<self.len and flag:
5                  flag = False
6                  for j in range(self.len-i):
7                          if self.list[j+1].key < self.list[j].key:
8                                  p = self.list[j]
9                                  self.list[j] = self.list[j+1]
10                                 self.list[j+1] = p
11                                 flag = True
12                 i += 1
```

3.2 快速排序

快速排序是一种分区交换排序算法，是冒泡排序的改进，其采用了分治策略，将问题划分成若干个规模更小但和原问题相似的子问题，然后用递归方法解决这些子问题，最终将它们组合成原问题的解。



快速排序将要排序的序列分成独立的两个部分，其中一部分的关键字值都比另一部分的关键字值大，然后分别对这两个部分进行快速排序，排序过程递归进行，整个序列最终达到有序。

独立的两个部分的划分方法为在序列中任意选取一条记录，然后将所有关键字值比它大的记录放到它的后面，将所有关键字值比它小的记录放到它的前面。这条记录叫支点。

3.2 快速排序

其主要步骤如下。

(1) 设置两个变量low、high，分别表示待排序序列的起始下标和终止下标。

(2) 设置变量p=list[low]。

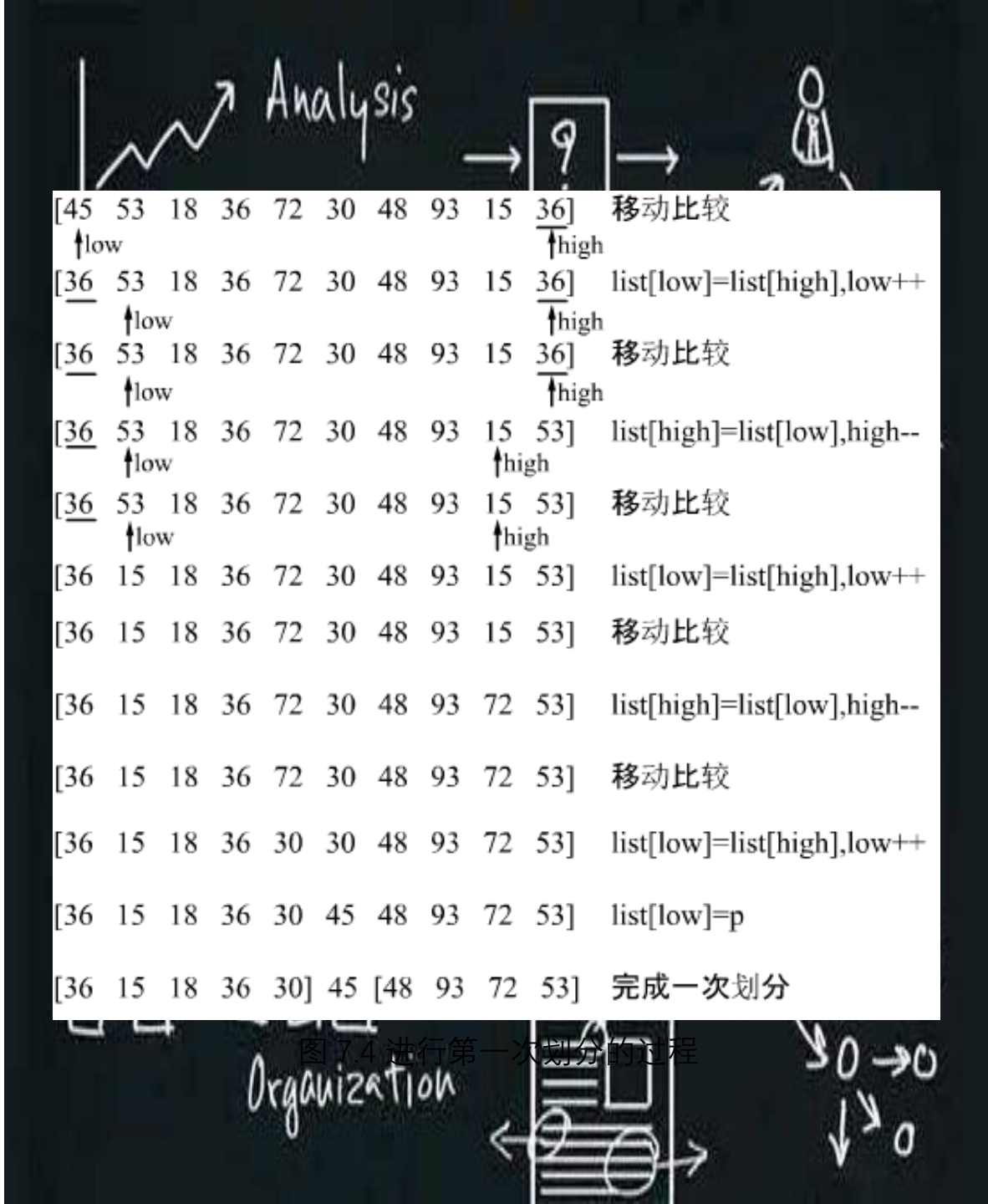
(3) 从下标为high的位置从后向前依次搜索，当找到第1个比p的关键字值小的记录时将该数据移动到下标为low的位置上，low加1。

(4) 从下标为low的位置从前向后依次搜索，当找到第1个比p的关键字值大的记录时将该数据移动到下标为high的位置上，high减1。

(5) 重复步骤(3)和(4)，直到high=low为止。

(6) list[low]=p。

假设一组待排序的记录的关键字序列为{45,53,18,36,72,30,48,93,15,36}



[45 53 18 36 72 30 48 93 15 36]	移动比较
[36 53 18 36 72 30 48 93 15 36]	list[low]=list[high],low++
[36 53 18 36 72 30 48 93 15 36]	移动比较
[36 53 18 36 72 30 48 93 15 53]	list[high]=list[low],high--
[36 53 18 36 72 30 48 93 15 53]	移动比较
[36 15 18 36 72 30 48 93 15 53]	list[low]=list[high],low++
[36 15 18 36 72 30 48 93 15 53]	移动比较
[36 15 18 36 72 30 48 93 72 53]	list[high]=list[low],high--
[36 15 18 36 72 30 48 93 72 53]	移动比较
[36 15 18 36 30 30 48 93 72 53]	list[low]=list[high],low++
[36 15 18 36 30 45 48 93 72 53]	list[low]=p
[36 15 18 36 30] 45 [48 93 72 53]	完成一次划分

图 7.4 进行第一次划分的过程

Python实现

```
1  def qSort(self,low,high):
2          if low<high:
3                  p = self.Partition(low,high)
4                  self.qSort(low,p-1)
5                  self.qSort(p+1,high)
6
7  def Partition(self,low,high):
8          p = self.list[low]
9          while low<high:
10                 while low<high and self.list[high].key>p.key:
11                         high -= 1
12                 if low<high:
13                         self.list[low] = self.list[high]
14                         low += 1
15                 while low<high and self.list[low].key<p.key:
16                         low += 1
17                 if low<high:
18                         self.list[high] = self.list[low]
19                         high -= 1
20         self.list[low] = p
21         return low
```

3.2 算法性能分析

- 时间复杂度



- 快速排序的执行时间与数据元素序列的初始排列以及基准值的选取有关。在最坏情况下待排序序列基本有序，每次划分只能得到一个子序列，等同于冒泡排序，时间复杂度为$O(n^2)$； 在一般情况下，对于具有n条记录的序列来说，一次划分需要进行n次关键字的比较，其时间复杂度为O(n)，设T(n)为对其进行快速排序所需要的时间，可得:

- $T(n) \leq cn + 2T\left(\frac{n}{2}\right)$
- $\leq cn + 2\left(\frac{cn}{2} + 2T\left(\frac{n}{4}\right)\right) = 2cn + 4T(\frac{n}{4})$



- $\leq 2cn + 4\left(\frac{cn}{4} + 2T\left(\frac{n}{8}\right)\right) = 3cn + 8T(\frac{n}{8})$
- ⋮
- $\leq cn\log_2 n + nT(1) = O(n\log_2 n)$
  - 所以快速排序是内部排序中速度最快的，其时间复杂度为$O(n\log_2 n)$。



  - 快速排序的基准值的选择有许多方法，可以选取序列的中间值等，但由于数据元素序列的初始排列是随机的，不管如何选择基准值总会存在最坏情况。总之，当n较大并且数据元素序列随机排列时快速排序是快速的； 当n很小或者基准值选取不合适时，快速排序较慢。

3.2 算法性能分析

空间复杂度

快速排序需要额外存储空间栈来实现递归，递归调用的指针的参数都要存放到栈中。快速排序的递归过程可用递归树来表示。在最坏情况下树为单枝树，高度为O(n)，其空间复杂度为O(n)。若划分较为均匀，二叉树的高度为O(log2n),其空间复杂度也为O(log2n)。



算法稳定性

快速排序是一种不稳定的排序算法。

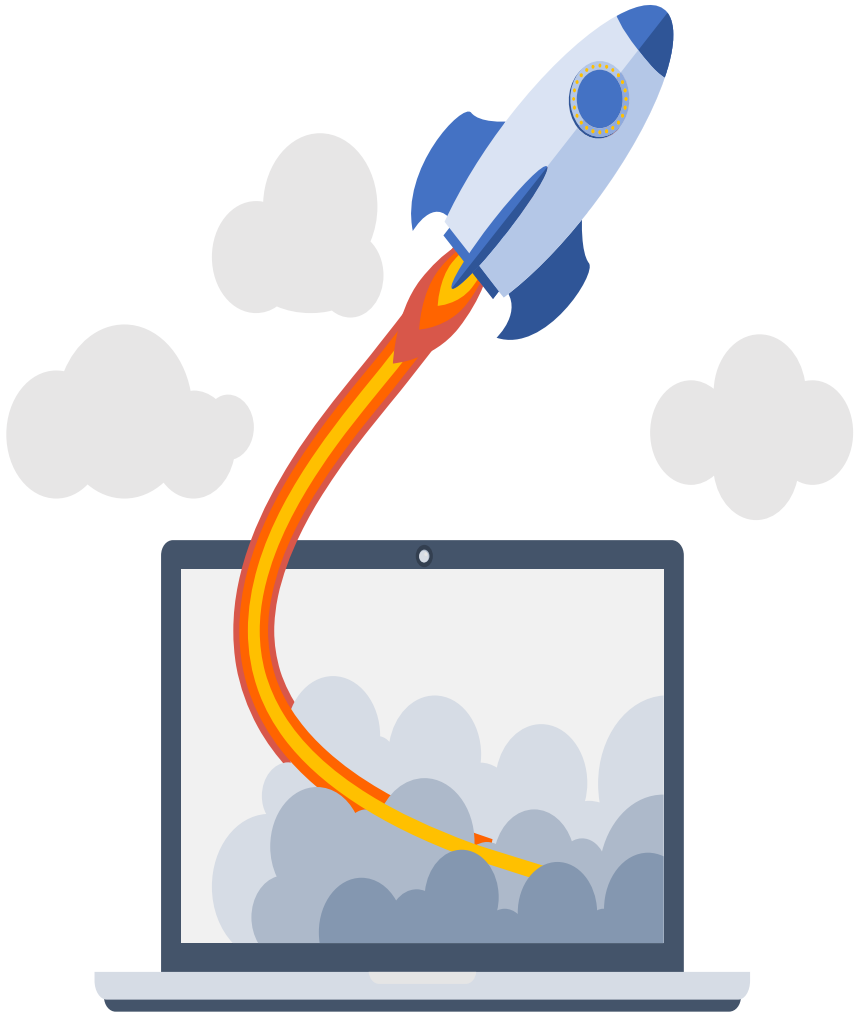
第四节 选择排序

- 直接选择排序
- 堆排序

4.1 直接选择排序

- 1. 直接选择排序算法的实现

- 直接选择排序是从序列中选择关键字值最小的记录进行放置，直到整个序列中的所有记录都选完为止。直接选择排序在第一次选择中从n个记录中选出关键字值最小的记录与第一个记录交换，在第二次选择中从n-1个元素中选取关键字值最小的记录与第二个记录交换，依此类推，在第i次选择中从n-i+1个元素中选取关键字值最小的记录和第i个记录交换，直到整个序列按关键字值有序时停止。



4.1 直接选择排序

其主要步骤如下。

(1) 令i=0。

(2) 在无序序列$\{a_i,a_{i+1},\ldots,a_{n-1}\}$中选出关键字值最小的记录$a_{min}$。

(3) a_{min}与a_i交换位置，i加1。

(4) 重复步骤(2)和(3)，直到i=n-2时停止。

假设一组待排序的记录的关键字序列为{36,23,18,56,78,70,45,2}，

直接选择排序的过程如图7.6所示。



Python实现

```
1  def selectSort(self):
2          for i in range(self.len-1): # 进行 n-1 趟选择
3                  tmp = i
4                  for j in range(i,self.len): # 寻找关键字值最小的记录的位置
5                          if self.list[j].key<self.list[tmp].key:
6                                  tmp = j
7                  # 交换位置
8                  p = self.list[i]
9                  self.list[i] = self.list[tmp]
10                 self.list[tmp] = p
```

4.1 算法性能分析

时间复杂度

直接选择排序的比较次数与数据元素序列的初始排列无关，移动次数与初始排列有关。直接选择排序的移动次数较少，最好情况为序列有序，移动0次，最坏情况为序列逆序，移动3(n-1)次。其比较次数较多，进行了n-1趟选择，每趟需要进行n-1-i次比较，所以总的比较次数为$\sum_{i=0}^{n-2}(n-i-1)=\frac{n(n-1)}{2}$，所以直接选择排序的时间复杂度为$O(n^2)$

空间复杂度

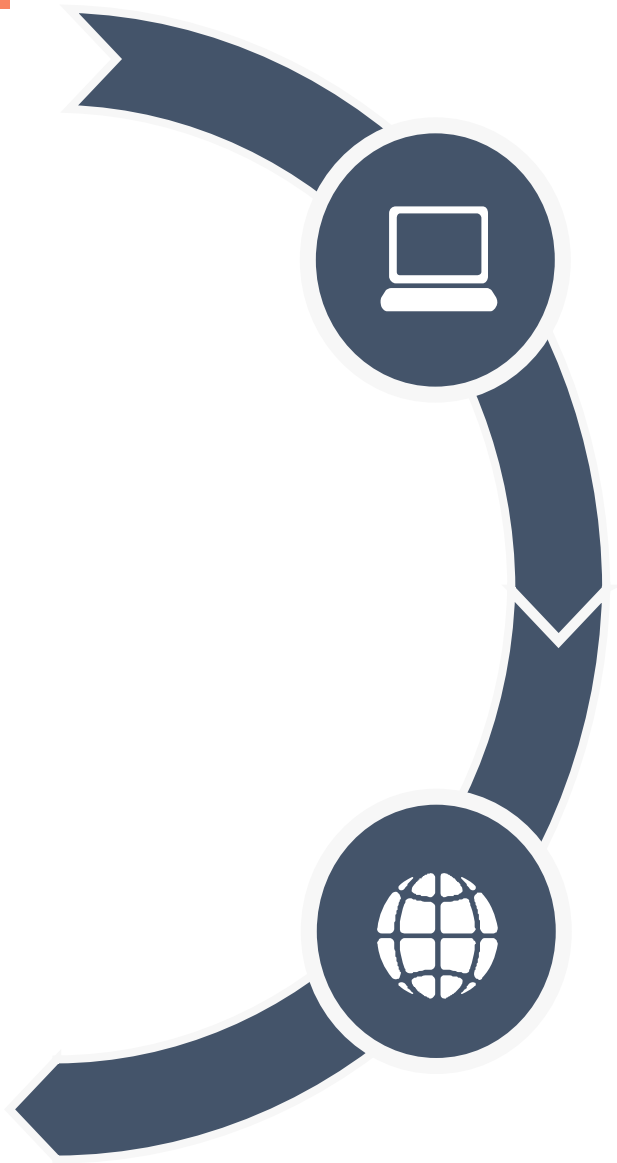
直接选择排序过程用了一个额外的存储单元p，所以其空间复杂度为O(1)。



算法稳定性

直接选择排序是一种不稳定的排序算法。

4.2 堆排序



1. 堆的定义

假设有 n 个记录关键字的序列为$\{k_0,k_1,\cdots,k_{n-1}\}$，当且仅当满足下面的条件时称为堆。

$$\begin{cases} k_i \le k_{2i+1}, 2i+1 < n \\ k_i \le k_{2i+2}, 2i+2 < n \end{cases}$$

或

$$\begin{cases} k_i \ge k_{2i+1}, 2i+1 < n \\ k_i \ge k_{2i+2}, 2i+2 < n \end{cases}$$

前者称为小顶堆，后者称为大顶堆。

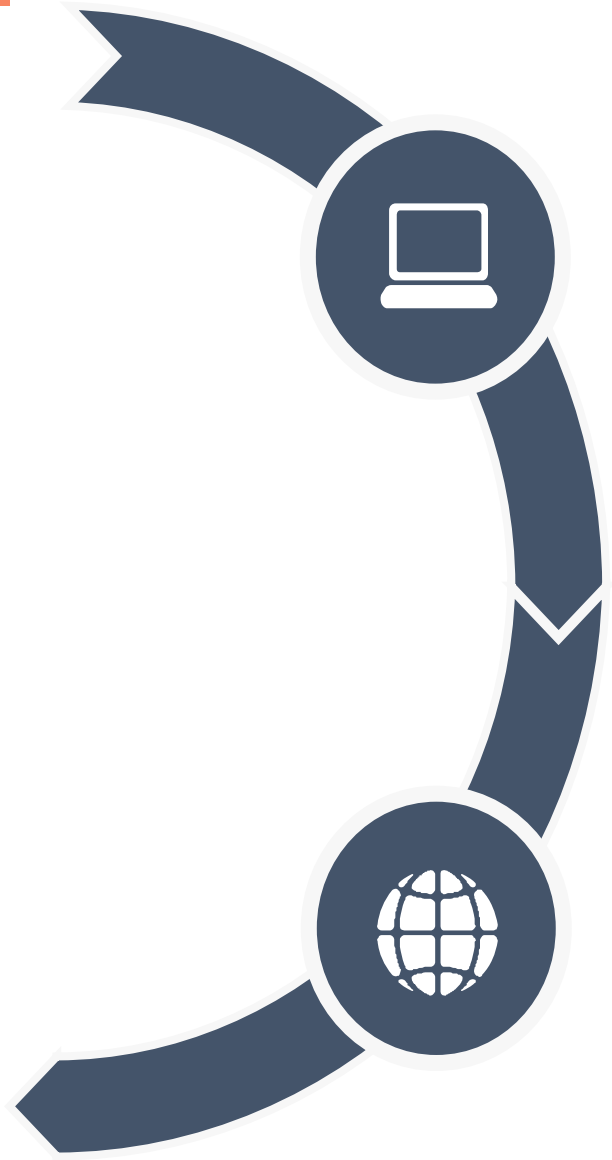
直接选择排序算法有以下两个缺点。

(1) 选择最小值效率低，必须遍历子序列，比较了所有元素后才能选出最小值。

(2) 每趟将最小值交换到前面，其余元素原地不动，下一趟没有利用前一趟的比较结果，需要重复进行数据元素关键字值的比较，效率较低。

堆排序是利用完全二叉树特性的一种选择排序。虽然堆中的记录无序，但在小顶堆中堆顶记录的关键字值最小，在大顶堆中堆顶记录的关键字值最大，因此堆排序是首先将 n 条记录按关键字值的大小排成堆，将堆顶元素与第 n-1 个元素交换位置并输出，再将前 n-1 个记录排成堆，将堆顶元素与第 n-2 个元素交换并输出，依此类推，即可得到一个按关键字值进行排序的有序序列。

4.2 堆排序



2. 用筛选法调整堆

在进行堆排序的过程中，当堆顶元素和堆中的最后一个元素交换位置后根结点和其子结点的关键字值不再满足堆的定义，需要进行调整。

用筛选法调整堆是将根结点和其左、右孩子结点的关键字值进行比较，其与具有较小关键字值的孩子结点进行交换。被交换的孩子结点所在的子树可能不再满足堆的定义，重复对不满足堆定义的子树进行交换操作，直到堆被建成。

调整堆的主要步骤如下。

(1) 设置变量 i 为需要调整的序列的最小下标 low，设置变量 j=2i+1,设置变量 p=list[i]。

(2) 当 j<high-1 时，若 list[j].key>list[j+1].key,j 加 1。

(3) 若 p>list[j].key,则 list[i]=list[j],i=j,j=2*i=1。

(4) 重复步骤(2)、(3)，直到 j≥high。

(5) list[i]=p。

用筛选法调整堆

```
1  def sift(self,low,high):
2          i = low
3          j = 2 * i + 1
4          p = self.list[i]
5          while j<high:
6                  if j<high-1 and self.list[j].key>self.list[j+1].key:
7                          # 比较左右孩子的关键字大小
8                          j += 1
9                  if p.key>self.list[j].key:
10                         # 交换父节点和子节点并相加进行筛选
11                         self.list[i] = self.list[j]
12                         i = j
13                         j = 2 * i + 1
14                 else:
15                         # 退出循环
16                         j = high + 1
17         self.list[i] = p
```

4.2 堆排序

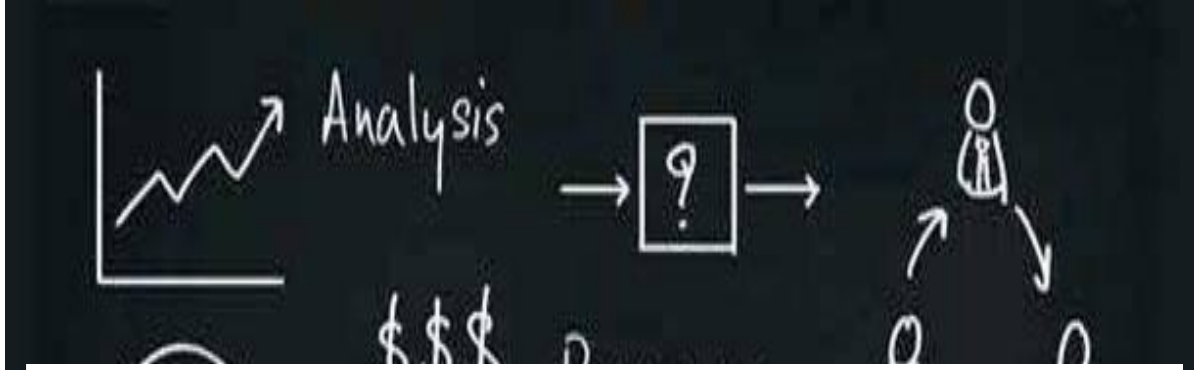
3. 堆排序

堆排序的主要步骤如下。

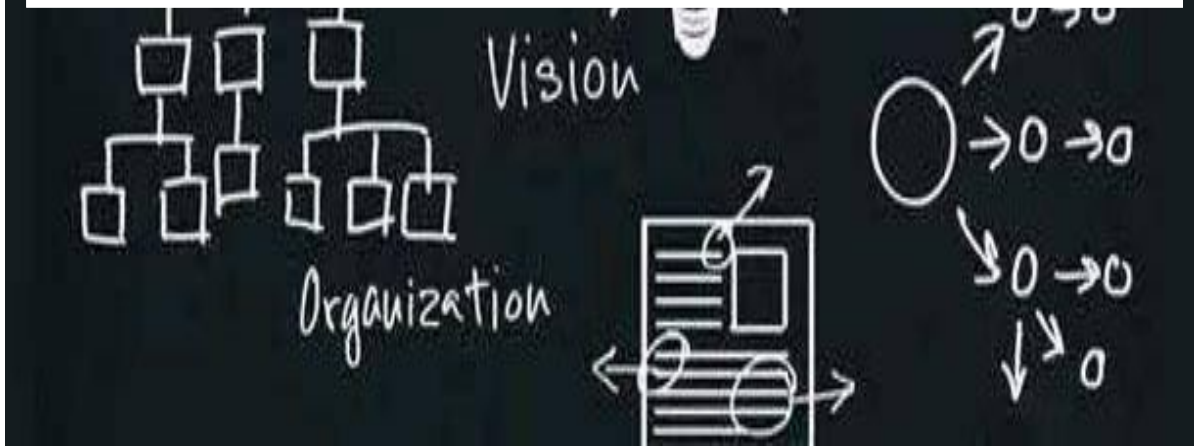
（1）将待排序序列建成一棵完全二叉树。（2）将完全二叉树建堆。

（3）输出堆顶元素并用筛选法调整堆，直到二叉树只剩下一个结点。

为一个序列建堆的过程就是对完全二叉树进行从下往上反复筛选的过程。筛选从最后一个非叶结点开始向上进行，直到对根结点进行筛选，堆被建成。




```
1  def heapSort(self):
2          for i in range(self.len//2-1,-1,-1): # 创建堆
3                  self.sift(i,self.len-1)
4          for i in range(self.len-1,0,-1): # 用筛选法调整堆
5                  p = self.list[0]
6                  self.list[0] = self.list[i]
7                  self.list[i] = p
8                  self.sift(0,i-1)
```



4.2 算法性能分析

时间复杂度

假设在堆排序过程中产生的二叉树的树高为k，则k=log2n＋1，一次筛选过程，关键字的比较次数最多为 2（k-1）次，交换次数最多为 k 次，所以堆排序总的比较次数不超过2（log2（n-1）＋log2（n-2）＋…＋log22）<2nlog2n，建初始堆的比较次数不超过4n 次，所以在最坏情况下堆排序算法的时间复杂度为O（nlogn）。

空间复杂度

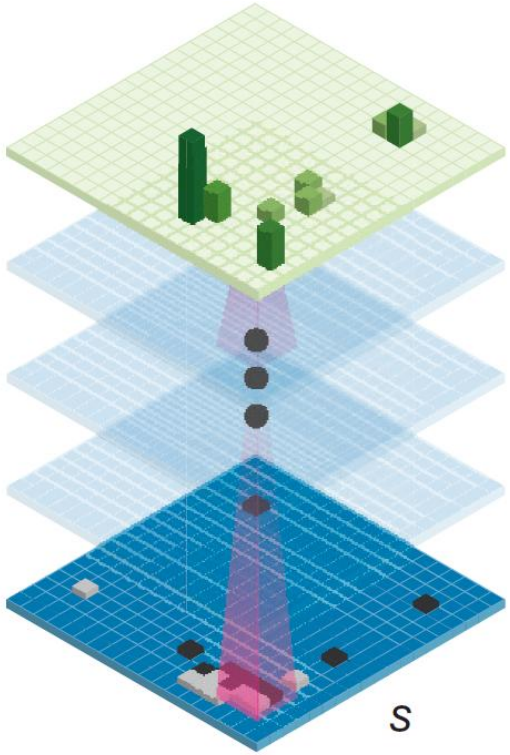
堆排序需要一个额外的存储空间p，其空间复杂度为O（1）

算法稳定性

堆排序算法是不稳定的排序算法。

第五节 归并排序

5. 归并排序



归并排序是指将两个或者两个以上的有序表合并成一个新的有序表，其中有序表个数为 2 的归并排序叫二路归并排序，其他的叫多路归并排序。

1. 两个相邻有序序列归并

两个有序序列分别存放在一维数组的 a[i..k]和 a[k+1..j]中，设置数组 order[]存放合并后的有序序列。归并排序的主要步骤如下。

(1) 比较两个有序序列的第 1 个记录的关键字值的大小，将关键字值较小的记录放入数组 order[]中。

(2) 对剩余的序列重复步骤(1)的过程，直到所有的记录都放入了有序数组 order[]。

假设一组待排序的记录的关键字序列为{45,53,18,36,72,30,48,93,15,36}，归并排序的过程如图 7.7 所示。

[45] [53] [18] [36] [72] [30] [48] [93] [15] [36]

[45 53] [18 36] [30 72] [48 93] [15 36]

[18 36 45 53] [30 48 72 93] [15 36]

[18 30 36 45 48 53 72 93] [15 36]

[15 18 30 36 36 45 48 53 72 93]

图 7.7 归并排序的过程

两个相邻有序序列归并

```
1  def merge(self,order,a,i,k,j):
2          t = i
3          m = i
4          n = k + 1
5          while m<=k and n<=j:
6                  # 将具有较小关键字值的元素放入 order[]
7                  if a[m].key<=a[n].key:
8                          order[t] = a[m]
9                          t += 1
10                         m += 1
11                 else:
12                         order[t] = a[n]
13                         t += 1
14                         n += 1
15         while m<=k:
16                 order[t] = a[m]
17                 t += 1
18                 m += 1
19         while n<=j:
20                 order[t] = a[n]
21                 t += 1
22                 n += 1
```

5. 归并排序

2.一趟归并排序:

一趟归并排序是指将待排序中的有序序列两两合并的过程，合并结果仍放在数组order[]中

```
1  def mergepass(self,order,a,s,n):
2          p = 0
3          while p+2*s-1<=n-1: # 两两归并长度均为 s 的有序表
4                  self.merge(order,a,p,p+s-1,p+2*s-1)
5                  p = p+2*s
6          if p+s-1<n-1: # 归并长度不等的有序表
7                  self.merge(order,a,p,p+s-1,n-1)
8          else: # 将一个有序表中的元素放入 order[]中
9                  for i in range(p,n):
10                         order[i] = a[i]
```

3. 二路归并排序

【算法 7.10】二路归并排序。

```
1  def mergeSort(self):
2          s = 1 # 已排序的子序列的长度，初始值为 1
3          order = [ None ] * self.len
4          while s<self.len: # 归并过程
5                  self.mergepass(order,self.list,s,self.len)
6                  s = s * 2
7                  self.mergepass(self.list,order,s,self.len)
8                  s = s * 2
```

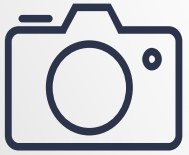
5 算法性能分析

时间复杂度

二路归并排序算法的时间复杂度等于归并的趟数与每一趟时间复杂度的乘积。归并的趟数为log2n，每一趟归并的移动次数为数组中记录的个数n，比较次数一定不大于移动次数，所以每一趟归并的时间复杂度为O（n），故二路归并排序算法的时间复杂度为O（nlog2n），



时间复杂度



空间复杂度



算法稳定性

空间复杂度

二路归并排序算法需要使用一个与待排序序列等长的数组作为额外存储空间存放中间结果，所以其空间复杂度为O（n）。

算法稳定性

二路归并排序算法是一种稳定的排序算法。

小结：排序方法比较

排序方法	平均时间	最坏情况	辅助空间	稳定性
直接插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
折半插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
直接选择排序	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
希尔排序	$O(n^{1.3})$	$O(n^{1.3})$	$O(1)$	不稳定
快速排序	$O(n\log_2 n)$	$O(n^2)$	$O(\log_2 n)$	不稳定
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	不稳定
二路归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	稳定

小结

- (1) 排序是指将一组数据按照关键字值的大小(递增或者递减)次序进行排列。按照排序过程中所涉及的存储器的不同可将排序分为内部排序和外部排序两种类型。排序又可分为稳定排序和不稳定排序。
- (2) 常用的内部排序算法有插入排序、交换排序、选择排序、归并排序。
- (3) 插入排序算法有两种，直接插入排序算法是将一条待排序的记录按照其关键字值的大小插入到已排序的记录序列中的正确位置，依此重复，直到全部记录都插入完成； 希尔排序是分组的直接插入排序。



小结

- (4) 在交换排序中，冒泡排序是两两比较待排序记录的关键字，如次序相反则交换两个记录的位置，直到序列中的所有记录有序； 快速排序是将要排序的序列分成独立的两个部分，其中一部分的关键字值都比另一部分的关键字值大，然后分别对这两个部分进行快速排序。

- (5) 在选择排序中，直接选择排序是从序列中选择关键字值最小的记录进行放置，直到整个序列中的所有记录都选完为止； 堆排序是将n条记录按关键字值的大小排成堆，将堆顶元素与第n-1个元素交换位置并输出，依此类推，即可得到有序序列。

- (6) 归并排序是指将两个或者两个以上的有序表合并成一个新的有序表，其中有序表个数为2的归并排序叫二路归并排序，其他的叫多路归并排序。



谢谢观看